

UNIVERSITY OF CLAUDE · FREE RESOURCE MAP

Resource Map

CSCE 605 Compiler Design — module-by-module mapping onto free university courseware

HOW TO USE THIS

This is your lecture hall. For each module: watch or read the mapped external material first, then work the module slides and notes, which are written to assume you have seen it.

Where a specific lecture number is given, that is the one that covers this module. Where a book chapter is given, read that chapter, not the whole book.

Everything here is free. Resources remain under their own licences — link to them, do not redistribute them.

Primary sources for the whole course

Resource	Role in this course
Stanford CS143 — Compilers (free, self-paced)	The primary source for Modules 02 through 06. Complete lectures, assignments, and a full compiler project. Free and well paced.
Bob Nystrom — Crafting Interpreters (free book)	The best free introduction in existence. Two complete implementations, every line explained. Read the whole thing early — it makes the rest of the course concrete.
Cooper & Torczon — Engineering a Compiler	The reference for Modules 07 through 11. Stronger than the Dragon Book on SSA, dataflow, and register allocation. Library copy.
LLVM — the Kaleidoscope tutorial and the Language Reference (free)	A working front end in a weekend, and then the IR specification that Module 07 is about. The Language Reference is worth reading as a document in its own right.
Khronos — the SPIR-V specification and SPIRV-Cross / DXC (free)	The Module 13 material. SPIR-V is an SSA IR you can read, which makes it an unusually good teaching artifact as well as a shipping one.
Muchnick — Advanced Compiler Design and Implementation	The exhaustive reference for Modules 08 and 09. Dense; use it to look things up rather than to read through.

Module-by-module map

Module 01 — What a Compiler Is

Resource	What it covers for this module
Nystrom — Crafting Interpreters (free book)	Read part II this week. A complete tree-walking interpreter, then a complete bytecode VM, with every line explained.
Stanford CS143 — Compilers, lecture 1 (free)	The phase structure of §1, with worked examples of what each one catches.
Chris Lattner — LLVM, in The Architecture of Open Source Applications (free)	The §1 reuse argument and the §2 IR argument , from the person who built the system that made both famous. Short.
Yang, Chen, Eide & Regehr — Finding and Understanding Bugs in C Compilers (free)	The Csmith paper of §4. Read the results section now and the method in Module 13.
Regehr — A Guide to Undefined Behavior in C and C++ (free)	The §4 hazard, with the deleted-null-check example worked through.

Module 02 — Lexical Analysis

Resource	What it covers for this module
Nystrom — Crafting Interpreters, 'Scanning' (free)	A complete hand-written lexer, explained line by line. The §3 recommendation, implemented.
Stanford CS143 — lexical analysis lectures (free)	Thompson's construction, subset construction, and minimisation, derived rather than asserted.
Russ Cox — Regular Expression Matching Can Be Simple And Fast (free)	The §1 blowup discussion, properly explained , including why general regex engines backtrack and lexers do not.
Unicode Consortium — UAX #31, Identifier and Pattern Syntax (free)	The identifier profile of §2's table. Short, and it saves inventing a rule badly.

Module 03 — Parsing I: Top-Down

Resource	What it covers for this module
Nystrom — Crafting Interpreters, 'Parsing Expressions' and 'Compiling Expressions' (free)	Recursive descent and then Pratt parsing, both implemented in full.
Matklad — Simple but Powerful Pratt Parsing (free)	The clearest explanation of §3 available , including the binding-power asymmetry that everyone gets wrong first.
Stanford CS143 — top-down parsing lectures (free)	FIRST, FOLLOW, and the LL(1) condition of §2, derived.
Rust — the rustc diagnostics guide (free)	§4 taken seriously by people who made it a priority. Worth reading for the standards it sets.

Module 04 — Parsing II: Bottom-Up

Resource	What it covers for this module
Stanford CS143 — bottom-up parsing lectures (free)	The item construction and the LR family of §2, worked through carefully.
Aho, Lam, Sethi & Ullman — Compilers (the Dragon Book), chapter 4	The definitive treatment of LR parsing. Library copy; this chapter is the book's strongest.
Bison manual — conflict reporting and resolution (free)	How to read a conflict report , which is the practically important skill of §3.
Tree-sitter — documentation and design (free)	Incremental, error-tolerant GLR parsing for editors — the §4 requirement, taken seriously and generated rather than hand-written.

Module 05 — Names, Scopes, and Symbol Tables

Resource	What it covers for this module
Nystrom — Crafting Interpreters, 'Resolving and Binding' and 'Closures' (free)	The two-pass resolver and closure capture, implemented and measured. The slot-index optimisation of §3 is here with numbers.
Stanford CS143 — semantic analysis lectures (free)	Symbol tables, scoping, and the structure of §2.
Cooper & Torczon — Engineering a Compiler, chapter 5	Symbol table implementation choices and their costs.
rustc dev guide — name resolution (free)	A production resolver handling modules, imports, macros, and cycles — the §2 complications at full scale.

Module 06 — Type Checking

Resource	What it covers for this module
Pierce — Types and Programming Languages	The standard reference. Chapters on simple types, subtyping, and variance cover §1 through §3. Library copy; the author's slides are free.
Stanford CS143 — type checking lectures (free)	The rule-to-code correspondence of §2, worked through.
Damas & Milner; and 'Write You a Haskell' (free)	Hindley–Milner implemented , with unification and generalisation in readable code.
Microsoft — HLSL reference; Khronos — GLSL specification (free)	The §4 restrictions, stated normatively. The storage-class and precision sections are the interesting ones.

Module 07 — Intermediate Representation and SSA

Resource	What it covers for this module
Cytron, Ferrante, Rosen, Wegman & Zadeck — Efficiently Computing SSA Form (free)	The original construction of §3 , including dominance frontiers. Still the clearest statement of the placement argument.
Cooper, Harvey & Kennedy — A Simple, Fast Dominance Algorithm (free)	The iterative dominator algorithm — simpler than Lengauer–Tarjan and faster on real CFGs. Four pages; implement this one.
Briggs, Cooper, Harvey & Simpson — Practical Improvements to the Construction and Destruction of SSA Form (free)	The §4 swap and lost-copy problems , named and solved by the people who found them.
LLVM Language Reference, and SSA Book (free)	A full free book on SSA, and the IR specification to read alongside it.

Module 08 — Dataflow Analysis

Resource	What it covers for this module
Cooper & Torczon — Engineering a Compiler, chapter 9	The framework of §1 and the analyses of §2 , with the termination argument done properly.
Kildall; and Nielson, Nielson & Hankin — Principles of Program Analysis	The lattice-theoretic foundations. Heavy going; useful for the monotonicity and fixpoint results.
Hind — Pointer Analysis: Haven't We Solved This Problem Yet? (free)	The §3 bottleneck, surveyed . The title is the conclusion.
Wegman & Zadeck — Constant Propagation with Conditional Branches (free)	Sparse conditional constant propagation — the §4 result that SSA makes an analysis both faster and more precise.

Module 09 — Optimisation

Resource	What it covers for this module
Cooper & Torczon — Engineering a Compiler, chapters 8 and 10	The catalogue of §1 with the correctness conditions for each transformation stated.
LLVM — passes documentation, and opt-print-after-all (free)	Run it on your own code this week . Watching the IR change pass by pass is worth more than any description of the passes.
Chandler Carruth — talks on LLVM optimisation and benchmarking (free)	Particularly the benchmarking material of §4 , including how microbenchmarks mislead and what to do instead.
Mytkowicz, Diwan, Hauswirth & Sweeney — Producing Wrong Data Without Doing Anything Obviously Wrong (free)	The code layout effect of §4 , measured. Link order and environment size alone moved results more than the optimisations being studied.

Module 10 — Code Generation

Resource	What it covers for this module
Appel — Modern Compiler Implementation, chapter 9 (instruction selection)	Tree tiling and the dynamic programming algorithm of §1 , presented clearly.
System V AMD64 ABI specification (free)	The §3 contract, normatively . The struct classification section is the one worth reading carefully.
Cooper & Torczon — Engineering a Compiler, chapters 11 and 12	Selection and scheduling, with the DAG complexity result.
DWARF Debugging Standard (free)	Location lists and inlining records — the §4 mechanism. Dense; read the introduction and the location-list chapter.

Module 11 — Register Allocation

Resource	What it covers for this module
Chaitin — Register allocation and spilling via graph colouring; Briggs et al. — Improvements to graph colouring (free)	The two papers of §1 . Briggs' optimistic colouring is the change worth understanding.
Poletto & Sarkar — Linear Scan Register Allocation (free)	The §3 algorithm , from its authors. Short and clear.
Hack, Grund & Goos — Register Allocation for Programs in SSA Form (free)	The chordality result of §4 , with the subtree argument.
Wimmer & Franz — Linear Scan Register Allocation on SSA Form (free)	Interval splitting and the production JIT version of §3.

Module 12 — Runtimes: Garbage Collection, JIT, and Scripting

Resource	What it covers for this module
Jones, Hosking & Moss — The Garbage Collection Handbook	The reference for §1 . Library copy; the authors maintain a free bibliography and the first chapters set up the trade-offs well.
Nystrom — Crafting Interpreters, 'Garbage Collection' (free)	A complete mark-sweep collector implemented and explained, including the root-finding problems that make it tricky.
V8 and SpiderMonkey engineering blogs (free)	Tiering, inline caches, and deoptimisation described by the people implementing them, with measurements. The best free source for §2.
Roberto Ierusalimschy et al. — The Implementation of Lua 5.0 (free)	How a language designed for embedding is built — the §3 option, from its authors. Short and unusually clear.

Module 13 — Shader Compilation, Testing, and Shipping

Resource	What it covers for this module
Khronos — SPIR-V specification, SPIRV-Cross, and SPIRV-Tools (free)	The §1 pipeline. spirv-dis and spirv-opt let you watch the offline half of it happen.
Yang, Chen, Eide & Regehr — Finding and Understanding Bugs in C Compilers (free)	Csmith. The method of §3, the results, and the CompCert comparison. The most important paper in this course.
Lee, Hur, Nagarakatte et al. — Alive2 (free)	Translation validation for LLVM passes, usable on your own transformations.
Leroy — CompCert, and Formal Verification of a Realistic Compiler (free)	The verified compiler of §3, with the papers explaining what exactly is proved and what is assumed.
Microsoft — DirectX Shader Compiler; and the Vulkan pipeline cache documentation (free)	The practical §1 and §2 material: PSO creation, pipeline caches, and specialisation constants.